

51单片机 Day1 笔记

一、学习路线总览

整个嵌入式学习分两条腿走路：

软件线

- C语言 → 数据结构 → Linux 软件编程（文件系统、多任务、网络、内存管理）

硬件线

- 51单片机：原理图、外设（LED、按键、蜂鸣器、中断、定时器、PWM、UART）、调试工具（万用表、示波器）
- ARM：裸机（汇编、GPIO、I2C、SPI、ADC、LCD）+ Linux驱动

今天我们主要讲硬件概念和 51 单片机入门。

二、核心芯片概念

CPU vs MCU vs MPU

类型	全称	特点	典型应用
CPU	Central Processing Unit	纯处理器，需要外接一切	PC、服务器
MCU	Micro Controller Unit	集成 CPU+RAM+ROM+GPIO+Timer+UART	简单控制，51单片机
MPU	Micro Processing Unit	只有 CPU 芯片，外接存储和外	复杂应用，跑 Linux

一句话区别：MCU 是把整个电脑做到一个芯片里，MPU 只是一个大脑，手脚得自己配。

GPU / NPU / FPU

- **GPU**（显卡核心）：图形处理、图像渲染，结构是 显存+GPU芯片+散热+输出接口
- **NPU**：神经网络处理器，专做 AI 推理、卷积运算
- **FPU**：浮点运算单元，通常在 CPU 内部

SOC

System On Chip，片上系统，把多个不同功能的芯片集成在一起。

三、51单片机基本情况

- 1980年 Intel 推出 MCS-51 系列，8051 架构

- 常见厂商: Atmel(AT89C51)、Philips(P89V51)、**STC 宏晶** (STC89C52/STC89C52RC)
- 架构位宽: **8位**
- 开发板: HC6800-MS (普中51-MS)
- 晶振: 12.0000MHz / 11.0592MHz
- 封装: 双列直插式 DIP-40 (40个脚)
- 40脚分4组: **P0、P1、P2、P3**, 每组8个脚

芯片发展脉络

ARM 只设计 CPU 方案, 授权给三星、Atmel、NXP、飞思卡尔等厂家去生产。比如 NXP 的 IMX6ULL 就是 ARM 架构

四、频率与时间——必须搞清楚

频率就是每秒振荡多少次, 和时间是倒数关系。

频率单位

$$1\text{KHz} = 10^3 \text{ Hz}$$

$$1\text{MHz} = 10^3 \text{ KHz} = 10^6 \text{ Hz}$$

$$1\text{GHz} = 10^3 \text{ MHz} = 10^9 \text{ Hz}$$

$$1\text{THz} = 10^3 \text{ GHz} = 10^{12} \text{ Hz}$$

大换小→乘, 小换大→除。

比如: $2.1\text{GHz} = 2.1 \times 10^9 = 2,100,000,000 \text{ Hz}$

每个周期时间 = $1/2,100,000,000 \approx 0.5\text{ns}$

时间单位

s → ms → us → ns (依次除1000)

1s = 1000ms = 1,000,000us = 1,000,000,000ns

五、内存和外存

特性	内存 (RAM)	外存 (ROM/硬盘)
速度	快	慢
掉电	数据丢失	数据保留
价格	贵 (内存条)	便宜 (硬盘、SD卡)
存什么	运行时的变量	程序代码

程序运行流程：外存 (a.out) → 加载到内存 → CPU 从内存取指令和数据执行。

ROM 和 RAM 在单片机里

- **ROM** (只读存储器)：放程序代码和指令，一次烧进去
- **RAM** (随机访问存储器)：放程序运行时产生的变量

```
int a = 2, b = 3;
int sum = a + b;
// CPU通过地址总线找a、b的地址 (RAM中)
// 数据总线读取a、b的值
// CPU运算
// 数据总线把sum写回RAM
```

六、CPU 与 RAM 交互——三大总线

总线	方向	作用
地址总线	CPU→RAM (单向)	CPU 发地址，找到数据在哪。8位线最大寻址 $2^8=256\text{byte}$
数据总线	双向	CPU 和 RAM 之间搬运数据
控制总线	双向	CPU 发读写指令

七、位运算——操作硬件的钥匙

8位数据: bit7(最高位 MSB) 到 bit0(最低位 LSB)

假设 `unsigned char t = 0;` (0000 0000)

按位或 `|` → 指定位置1

将 bit0 置1: `t = t | (1 << 0)` → 0000 0001

将 bit7 置1: `t = t | (1 << 7)` → 1000 0000

将 bit2和bit3 同时置1:

`t |= (1 << 2)` → 0000 0100

`t |= (1 << 3)` → 0000 1100

或者一步到位: `t |= (1 << 2) | (1 << 3)`

规律: 想把第 n 位置1, 就 `t |= (1 << n)`

按位与 & → 指定位清0

```
t = 0xFF (1111 1111)
```

```
将 bit1 清0: t &= ~(1 << 1) → 1111 1101
```

```
将 bit2 清0: t &= ~(1 << 2) → 1111 1011
```

规律：想把第 n 位清0，就 `t &= ~(1 << n)`

其他运算符

运算符	说明	口诀
<<	左移	低位补0，高位丢弃
>>	右移	高位补0（无符号）
^	异或	相同得0，不同得1
~	取反	1变0，0变1

八、外设寄存器——软件控制硬件的桥梁

外设寄存器就是一段有固定地址的内存空间，写不同的值进去就能控制硬件行为。

```
#define P2 *((unsigned char *)0xA0)
```

```
P2 = 0xFF; // 1111 1111 → 全灭/全高电平
```

```
P2 = 0x00; // 0000 0000 → 全亮/全低电平
```

往 P2 这个地址写数据，就会反映到 P2 组那 8 个引脚的电平状态上。

九、电阻标号

数码标注法：前几位是有效数字，最后一位是10的几次方。

$$102 = 10 \times 10^2 = 1\text{K}\Omega$$

$$103 = 10 \times 10^3 = 10\text{K}\Omega$$

十、数码管动态显示

开发板搭载 **4位共阴极数码管**。

显示流程

1. **位选** (P10-P13)：通过 NPN 三极管选通哪一位数码管
2. **段选**：给对应引脚高电平点亮该段 (a-g + dp)

动态显示原理——视觉暂留

依次快速刷新每位数码管，人眼来不及反应就觉得四位数同时在显示：

选第1位→显示值→delay(小延时)
选第2位→显示值→delay(小延时)
选第3位→显示值→delay(小延时)
选第4位→显示值→delay(小延时)
→ 循环

速查

要做的		怎么写
第n位置1	<code>`t \</code>	<code>= (1 << n)`</code>
第n位清0	<code>`t &= ~(1 << n)`</code>	
翻转第n位	<code>`t ^= (1 << n)`</code>	
整个端口全亮	<code>`P2 = 0x00` (共阴极低电平亮)</code>	
整个端口全灭	<code>`P2 = 0xFF`</code>	

网络编号：相同编号的引脚在实际电路中就是一根导线，画原理图时用它避免连线太乱。